3 Distributed coordination

Contents

3.1 Introduction

3.2 Distributed mutual exclusion

3.2.1 Centralized algorithms

3.2.2 Distributed algorithms

3.3 Election algorithms

3.3.1 Bully algorithm

3.3.2 Ring algorithm

3.4 Group communication

3.4.1 System model

3.4.2 Multicast semantics

3.4.3 Group membership management

3.4.4 Examples of group communication systems

3.5 Replication

3.5.1 System model

3.5.2 Consistency criteria

3.5.3 Replication techniques

3.6 Distributed transactions

3.6.1 Execution of distributed transactions

3.6.2 Atomic commitment

3.6.3 Recovery mechanisms

3.7 The Consensus problem

3.8 Exercises

3.1 Introduction

In the previous chapter we have dealt with the fundamental problems and concepts related to the management of the global state in a distributed system. We have seen that the practical limitations in the synchronization of local clocks led us to consider a logical time model that allows to express the causality between events and to introduce a definition of consistent global state from the concept of cut. On this basis we will describe here a series of coordination problems posed by distributed applications.

- Mutual exclusion. Exclusive access to shared resources, a classic problem of coordination/synchronization in centralized systems, when transferred to a distributed system requires a new approach, which involves communication among the participating processes to agree on the order of access.

- Election. The figure of a coordinating process or leader is frequent in distributed systems. The possibility of failure of the coordinator makes necessary mechanisms that allow the promotion of a process as a new coordinator, which implies detecting the failure of the coordinator and the election of a new one.

- Group communication. The multicast or diffusion of messages to a group of processes requires defining criteria of reliability and order in the delivery of messages and implementing algorithms that guarantee them.

- Replicated services. When a set of redundant servers try to provide fault tolerance or high availability, we need models that provide the adequate consistency for updating the replicas and for managing the joins and leaves.

- Distributed transactions. In reservation or banking systems, the update of the distributed state by means of an operation that involves several processes (transaction) must preserve invariants that imply agreement among the participating processes on the result of the operation.

Although the presented applications appear to be of a different nature, they coincide in terms of context (processes that update the distributed global state and that are subject to failure) and in terms of the objective (the coordination among the processes for a consistent update of the global state). [COU05] is the most general bibliographic reference for this topic.

3.2 Distributed mutual exclusion

More limited than in centralized operating systems, mutual exclusion is necessary in distributed systems to resolve particular situations in accessing shared resources, such as distributed file systems (for example, in NFS[1]), distributed shared memory, and Input/Output (for example, window systems). Distributed mutual exclusion algorithms can be centralized or distributed.

3.2.1 Centralized algorithms

They are based on the existence of a coordinating process of the critical section, which manages requests to enter and leave the critical section.

- Request to enter the critical section:
  - The requesting process sends a message to the coordinator and waits for a confirmation message.
  - If the critical section is free, the coordinator replies with a confirmation message.
  - If it is occupied, it does not reply (or replies with a denial message), and enqueues the request.
- Request to leave the critical section:
  - Send a message to the coordinator indicating the leave.
  - If the queue of pending processes is not empty, the coordinator chooses the first to enter and sends a confirmation message, removing it from the queue.

This algorithm guarantees mutual exclusion, no deadlock and no starvation. The problem with centralized algorithms is that the coordinator becomes a bottleneck, so they are not very scalable. In addition, they offer little tolerance to failures, since a failure in the coordinator leaves the clients without access to the critical section. On the other hand, an explicit treatment of client failures is required, since a client that does not request to leave the critical section will prevent the rest of clients from entering it.

[1] In UNIX, NFS uses a daemon process, *lockd*, which manages exclusive access to files.

3.2.2 Distributed algorithms

Unlike the centralized ones, in the distributed algorithms there is no differentiated process that coordinates the access to the critical section, but a protocol is implemented that allows reaching an agreement among the processes to decide which process enters the critical section.

3.2.2.1 Algorithm of Ricart and Agrawala

The algorithm of Ricart and Agrawala is based on the fact that any process that wants to enter the critical section must first obtain the approval of the rest of processes. It requires establishing a total order in the events, which is achieved with timestamps and assigning a predetermined order to processes with the same timestamp (for example, using the process identifier).

- Request to enter the critical section:
  - The requesting process sends to the rest of processes a message with its name, the critical section and a timestamp.
  - A receiver:
    - If it is not in the critical section, it sends a confirmation message to the requester.
    - If it is in the critical section, it does not reply (or denies) and enqueues the request.
    - If it is waiting to enter the critical section, it compares the requester's timestamp with the timestamp of its own request. If the requester's timestamp is older, it sends the confirmation message to the requester. If not, it does not reply (or denies) and it enqueues the request.
  - To enter the critical section, a process has to receive confirmations of all the processes.
- Request to leave the critical section:
  - The leaving process sends a confirmation message to all the processes it has in its queue, removing them from the queue.

The denial through non-reply offers zero fault tolerance (if one process fails, no process enters the critical section). The explicit denial allows using timeouts to

suppose that a process that does not reply has failed and to exclude it. However, it must be taken into account that an erroneous suspicion could lead to the violation of mutual exclusion in the access to the critical section. On the other hand, increasing the durations of the timeouts to increase the certainty of the failure hypothesis implies greater delays, with the consequent loss of performance.

If the set of processes is dynamic, it requires managing the joins and leaves of processes. As we will see, the support for group communication, if available, facilitates the implementation of this type of algorithms. The complexity of the algorithm, measured in terms of the number of messages required, is high, although it improves if *multicast* support is available.

With respect to centralized algorithms, in this algorithm all processes are bottlenecks, so it has no practical interest for a high number of processes.

3.2.2.2 Ring algorithm (*token ring*)

A logical ring is constructed by ordering the N processes circularly. Each process P_k has a predecessor P_{k-1} and a successor $P_{(k+1) \bmod N}$.

A token circulates one-way through the ring, by message passing, according to the following protocol:

- Initially, the token is given to a predefined process.
- A process P_k out of the critical section, upon reception of the token, sends it to process $P_{(k+1) \bmod N}$.
- When a process P_k wants to enter the critical section, it waits to receive the token. When it receives it, it keeps the token and enters the critical section.
- When a process P_k leaves the critical section, it sends the token to process $P_{(k+1) \bmod N}$.

This algorithm has some issues. On the one hand, we must look for a compromise between latency and traffic overload in the network. On the other hand, the ring is broken with the failure of a process. Fault tolerance can be improved if acknowledgment of the successor is required upon receiving the token. If the successor P_{k+1} does not reply (within a time period), P_k considers that P_{k+1} has failed and sends the token to P_{k+2}. Another issue is the loss of the token due to the failure of the process that has it.

Again, this type of algorithm can only work in synchronous system models, that is, under the assumption that, if a timeout expires without receiving an expected message, then the sender has failed. In this algorithm, when the token is not received within a deadline, it is necessary to initiate a protocol for the introduction of a new token. But since, in general, it is not possible to bound the time that each process retains the token (that is, the duration of the critical sections), it can happen that a presumably lost token continues to circulate, which implies managing scenarios with more than one token in the ring (by the use of token identifiers).

3.3 Election algorithms

Many distributed applications and services are based on the existence of a differentiated process that coordinates the work of a set of processes. For example, we have just seen that the centralized algorithm for distributed mutual exclusion requires a coordinating process. Moreover, in this chapter we will see the same need for the atomic commitment algorithms and in some models of replication.

In all these situations, it is necessary to detect if the coordinating process fails and to elect a new process that takes the role of coordinator. The election requires agreement on who will be the new and unique coordinator. Again, decisions are based on the existence of time bounds for receiving reply messages.

On the other hand, to determine the criteria for electing a process as a coordinator, it is necessary to define a total order among the set of processes. Hence, we will assume that the processes have associated unique identifiers according to which they can be ordered.

3.3.1 Bully algorithm

The bully algorithm, proposed by Garcia-Molina, is based on the idea that a process that suspects the failure of the coordinator tries to promote itself as the new coordinator by *challenging* the processes stronger than itself (i.e., with a higher identifier).

- When a process suspects that the coordinator has failed, it initiates an election and becomes *eligible*. An eligible process p executes the following protocol:

1 *p* sends an election message (a *challenge*) to all processes with a higher identifier than itself.

2 If nobody replies, *p* wins the election and becomes the new *coordinator*, spreading a message to let all processes know that it is the new coordinator. Otherwise, i.e., if *p* receives a message (from a process with a higher identifier), then it will not be the new coordinator and it will no longer be eligible.

- When a process *p* receives an election message (from a process *q* with a lower identifier):

  1 *p* replies by sending a *confirmation message* to *q*.

  2 If it was not already, *p* becomes *eligible* and executes the previous protocol.

In the end there will be a single eligible process, which will be the new coordinator, and which will make its new status known to all the processes.

3.3.2 Ring algorithm

The N processes are ordered circularly, each with a successor. The algorithm has two phases:

Phase 1:

- When a process P_i suspects that the coordinator has failed, it sends an *election message* to its successor $P_{(i+1) \bmod N}$ that contains P_i's identifier. If P_{i+1} does not reply, P_i repeats the sending to P_{i+2} and so on until it finds a process that acknowledges the reception of the election message
- When a process receives an election message, it forwards it to its successor, including in the message its own identifier, by applying the previous procedure.

Phase 2. Eventually the election message will reach P_i:

- The election message (that contains a list of all the processes not suspected to have failed) becomes a *coordinator message*, with the same content.
- After a complete round of the ring, the coordinator message will have informed all the processes that the new coordinator is the process with the highest identifier included in the message.

In another version of the algorithm [TAN07], the election message contains only the identifier of the eligible process, and a process modifies it only if its identifier is higher.

3.4 Group communication

In the previous chapter we considered a communication model by message passing between pairs of processes where the determination of causality was the fundamental problem to solve for the maintenance of the global state. However, in distributed systems, 1:*N* communication, which we will call *broadcast*/*multicast* or diffusion of messages to a group, introduces new concepts and the need to extend the model. There is a range of situations where multicast communication is applicable, with different requirements regarding its semantics. Examples of these applications are event notification; the registration or search of remote services, and the replication of services, either due to fault tolerance or availability and performance.

In general we will consider an infrastructure (*middleware*) for the adequate management of group communication, with a concrete interface and a precise semantics. The main characteristics that must be defined in a group communication system:

- The identification of groups of processes. The group of processes must be accessible as such, not as the sum of its members.
- If the groups are *open* or *closed*. In a closed group, only group members can broadcast messages to the group, while in an open group a process not belonging to the group can also broadcast messages to the group.
- The delivery of messages to the group, whose semantics will be determined by a set of properties.
- The management of the group membership. The leaves (in general, due to failure) and the joins (due to recovery) must be managed consistently.

3.4.1 System model

We will define a group communication model [COU05 §12] [MUL93 §5] where a group G consists of N processes P_1, P_2, …, P_N. The definition of groups as closed is not a restriction, since the diffusion of a message from outside the group can be modeled as a message *sent* to a group member that acts as a front-end and *broadcasts* the message to the rest of the processes of the group.

We will define the following primitives for the dissemination and delivery of messages to the group:

broadcast*(G, m)*. Broadcasts message *m* to the group *G*.

deliver*(m)*. Delivers message *m* to the process executing the primitive.

These primitives are implemented on top of the communication interface of the operating system, and are built by means of protocols that use the *send()* and *receive()* primitives. The broadcast can have hardware support (for example, with *IP multicast*). The structure of this model is represented in Figure 3.1.

We will consider that processes can fail. We will refer to a process that is not suspected of having failed as a **correct process**. The failure model is that of *crash* or stop failure, that is, a faulty process does not recover[2]. In our model, we will consider that communication channels are *quasi-reliable*, which means that a correct process will end up receiving the message sent by another correct process[3].

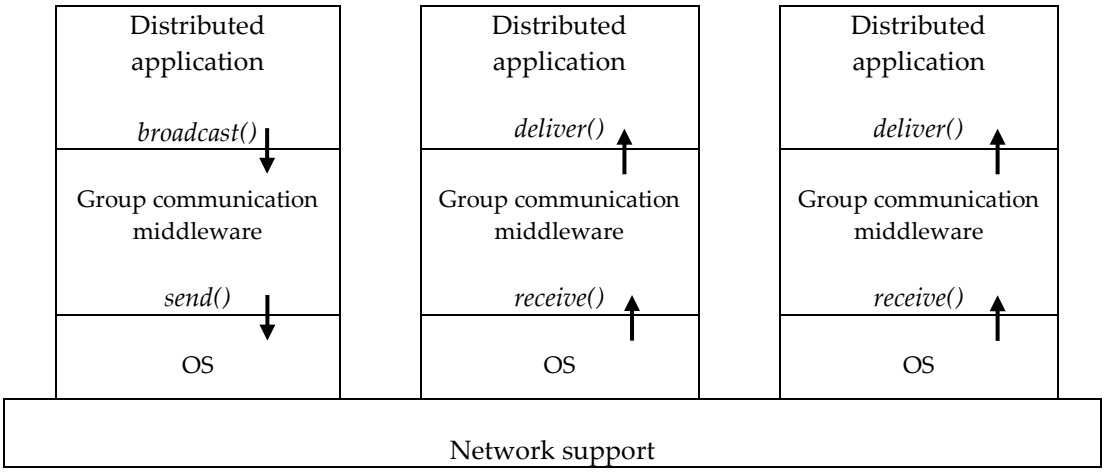


Figure 3.1. Structure of a group communication system.

[2] There are other failure models (*crash-recovery*, omission, Byzantine), which we will not consider here.

[3] If a process fails, the process is not correct. A *reliable* communication channel would guarantee the reception by a correct process of all messages sent to the channel even in case the sender is incorrect and fails right after sending the message.

3.4.2 Multicast semantics

To define the diffusion semantics in a group, we will consider two characteristics of group communication: the **reliability** of the diffusion and the **order of delivery** of the messages.

Let us consider first a simple implementation of the *broadcast* and *deliver* primitives:

- When a process P_i executes *broadcast(G, m)*:

  For all $P_j \in G$, *send*(P_j, *m*).

- When a process P_i executes *receive(m)*:

  deliver(*m*).

Such an implementation provides **unreliable broadcast**. In the following sections we will formally define reliable broadcast and introduce different message delivery orders. Intuitively, we will say here that broadcast is not reliable when the message broadcast to the group can be delivered by some correct processes and not delivered by other correct processes. Indeed, given the previous algorithm, it can be seen that if the process that broadcasts the message fails while executing *broadcast()* potentially some processes will receive the message and others will not.

3.4.2.1 Reliable broadcast

We will define as **reliable broadcast** the one that fulfills the following three properties:

- **Validity**. If a correct process P_i broadcasts a message *m*, then P_i eventually delivers *m*.

- **Agreement**. If a correct process delivers a message *m*, then all correct processes eventually deliver *m*.

- **Integrity**. For any message *m*, every process delivers *m* at most once, and only if *m* was previously broadcast.

The combination of these three properties is known as **all-or-none property**, which implies that a message broadcast to a group of processes, either reaches all the correct processes of the group or does not reach any correct process.

It can be verified that the unreliable broadcast defined in the previous section fulfills the validity and integrity properties, but not the agreement property.

The reliable broadcast will be expressed by the *R-broadcast()* and *R-deliver()*[4] primitives, and can be implemented on top of the unreliable *broadcast()* primitive with the following algorithm:

- When a process P_i executes *R-broadcast(G, m)*: *broadcast(G, m)*.
- When a process P_j executes *receive(m)*, if *m* has not been previously received:
  1. If $P_j \neq P_i$, *broadcast(G, m)*.
  2. *R-deliver(m)*.

It can be shown that this algorithm provides reliable broadcast. Obviously, the algorithm preserves the validity of the unreliable one. Also, if there is a correct process that delivers *m*, then it has been previously forwarded to all processes, so that by definition of quasi-reliable channel and extending the reasoning, in that case all the correct processes will end up receiving *m*, and therefore delivering it, providing agreement. Integrity is ensured because a process only executes the delivery the first time it receives the message, and only broadcast messages are delivered.

3.4.2.2 Message delivery order

Reliable broadcast does not provide any guarantee about the delivery order of messages. On top of the reliable broadcast different semantics can be implemented regarding the message delivery order:

- **FIFO order**. If a process broadcasts a message m_1 before a message m_2, then no correct process delivers m_2 if it has not previously delivered m_1.
- **Causal order**. If the broadcast of a message m_1 *occurs before* the broadcast of a message m_2, then no correct process delivers m_2 if it has not previously delivered m_1.
- **Total order**. If two correct processes P_i and P_j deliver two messages m_1 and m_2, then P_i delivers m_1 before m_2 if and only if P_j delivers m_1 before m_2.

[4] The prefix *R-* comes from *Reliable*.

Note that the causal order is stronger than the FIFO order, since the relationship *occurs before* includes the temporal order relationship between events of the same process. The **total order broadcast**[5] is the basis to ensure consistency: all the correct processes see the same sequence of messages[6]. Reliable broadcast is combined with different message delivery orders to give rise to different semantics, as shown in Table 3.1[7].

	FIFO order	Causal order	Total order
Reliable broadcast			
FIFO broadcast	•		
Causal broadcast	•	•	
Total order broadcast			•
Causal and total order broadcast	•	•	•

Table 3.1. Different semantics of reliable broadcast depending on the delivery order of messages.

In general, a broadcast algorithm with a given semantics can be constructed from an algorithm that provides a weaker semantics, as shown in [MUL93 §5]. For example, FIFO broadcast can be constructed on top of reliable broadcast by numbering the messages in the sender. Causal broadcast is achieved by including time vectors. It should be noted, however, that while the semantics of reliable, FIFO and causal broadcast do not require any assumption about time, the total order broadcast does require. In this way, as shown by Fischer, Lynch and Paterson, in asynchronous systems, where the transmission time of the messages and the relative speeds of execution of the processes cannot be bounded, a deterministic solution of the problems that require a distributed *consensus* is not possible, as it is the case of total order broadcast. We will deal with the **problem of Consensus** later on.

For synchronous systems there are different algorithms that implement total order broadcast, which rely on reliable broadcast. When internal synchronization is available, the following algorithm can be used. We will

[5] Historically called *atomic broadcast*.

[6] In theory, non-causal total order broadcast is possible, so we should distinguish between total consistency and causal consistency. However, in practice it is straightforward to obtain causal order when there is total order, so we can simply talk about consistency.

[7] Note that the combination of total order and non-causal FIFO order is not possible (see Exercise 3).

assume that message m broadcast in *R-broadcast* is labeled with the local timestamp, $t(m)$[8].

- When a process P_i executes *TO-broadcast(G, m)*:

  R-broadcast(G, <t(m), m>).

- When a process P_j executes *R-deliver(m)*:

  P_j schedules *TO-deliver(m)* for the local time instant $t(m)+\Delta$

Δ is a constant that is configured according to different parameters of the system, such as the maximum delay of messages, the maximum number of processes that can fail and the guarantees offered by the internal synchronization of the clocks [MUL93 §5]. The scheduling of the delivery in $t(m)+\Delta$ implies that if the local time instant in which *R-deliver(m)* is executed is after that time, the total order delivery cannot be carried out[9]. This algorithm also provides causal order.

Among the algorithms that do not use synchronized physical clocks, we can distinguish between centralized ones, which are based on a *sequencer*, and distributed ones, which are based on agreements between the processes that receive the messages[10].

We will describe here an algorithm based on sequencer. The sequencer, Q, which can be one of the processes of the group, G, or a specific process, decides the order in which the messages are delivered. Each process sends a copy of the message m to the sequencer, including a unique identifier *id(m)*. The sequencer assigns them a sequence number and communicates the delivery order to the processes of the group using a special message (*ORDER*). Each process $P_i \in G$ executes the following tasks. Initially the variable S_i has value 0.

- When P_i executes *TO-broadcast(G, m)*:

  R-broadcast(G $\cup$ *{Q}, <m, id(m)>)*

- When P_i executes *R-deliver(<m, id(m)>)*:

  Enqueues *<m, id(m)>* in the total order delivery waiting queue.

[8] This type of broadcast is called **timed reliable broadcast** [MUL93].

[9] Note that if the delivery was made, the total order could be violated. The alternative is to not TO-deliver the message and consider that the process is not correct, which indicates that the assumptions for the calculation of Δ were not correct.

[10] The ISIS total order broadcast protocol is an example of a distributed algorithm.

- When P_i executes *R-deliver(<ORDER, id, S>)*:
  1. P_i waits until *<m, id>* is in the queue and S is equal to S_i+1
  2. P_i executes *TO-deliver(m)*
  3. P_i removes *<m, id>* from the queue
  4. $S_i = S$

The sequencer Q executes the following task (initially S_G has value 0):

- When Q executes *R-deliver(<m, id(m)>)*:
  1. $S_G = S_G + 1$
  2. Q executes *R-broadcast(G, <ORDER, id(m), S_G>)*

3.4.3 Group membership management

The groups of processes can be static or dynamic. This last case is the most general and involves considering in the management of the group of processes the fact that the processes can leave the group (if they fail) and join it (once they recover from the failure)[11]. We will consider two operations on a group, which are implemented on top of the message passing interface (Figure 3.2):

join(P_i, G). Joining of a process P_i to the group G.

leave(P_i, G). Leaving of a process P_i from the group G.

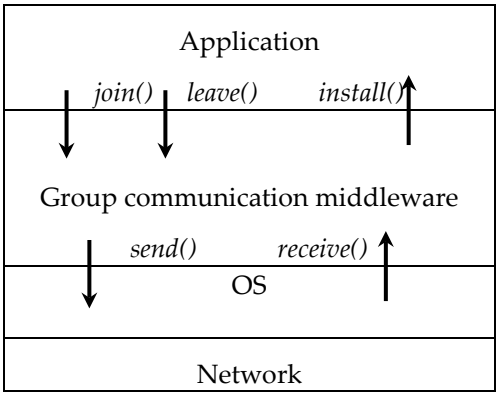


Figure 3.2. Structure of a group membership management system.

[11] It should be noted that there are two alternatives: (a) a failed process recovers with the same identity, and (b) a failed process does not recover, but a new one is created.

It should be noted that a process can be excluded from the group because it is suspected of having failed, which implies implementing an effective failure detection mechanism.

The group membership management is modeled around the concept of **view** v_k of the group as part of its global state. When there is a change in the group membership (by *join()*, by *leave()*, or by a failure), a new view v_{k+1} is installed by means of an ***install***(v_{k+1}) primitive that the group communication middleware provides to the application.

A view is defined by the set of processes that belong to the group when the view is installed. To guarantee the consistency between the installation of views and the delivery of messages, a total order is required. Consistent view management is defined by the following properties, as an extension of those that define reliable broadcast:

- **Termination**. The failure of a process or the execution of *join()* or *leave()* leads sooner or later to the installation of a new view.
- **Agreement**. All the correct processes deliver the same set of messages in a given view.
- **Validity**. Let P_i be a correct process that delivers a message m in a view v_k. If some process $P_j \in v_k$ does not deliver m in v_k, then P_j will not be in the view v_{k+1} installed by P_i.

This model of group communication with view management is called **virtually synchronous**, a term originally used in the ISIS system.

The group membership management is also a consensus problem.

3.4.4 Examples of group communication systems

The first middleware supporting group communication was ISIS[12], developed at the Cornell University. It includes a set of tools for building distributed applications on UNIX. ISIS derived in Horus, and this in turn in Ensemble[13].

ISIS provides a set of broadcast primitives and group membership management, mainly synchronous:

[12] http://www.cs.cornell.edu/Info/Projects/ISIS/
[13] http://www.cs.cornell.edu/Info/Projects/Ensemble/

- FBCAST. Provides reliable FIFO broadcast.
- CBCAST. Provides causal broadcast. It basically follows the algorithm of time vectors.
- ABCAST. Provides causal and total order broadcast (*atomic broadcast*).
- GBCAST. Group membership management. It is based on the concept of virtual synchrony. It uses total order broadcast, as ABCAST.

In another line, the ideas of ISIS have derived in a middleware based on free software and built in Java, JGroups[14] [BAN03]. JGroups provides a modular architecture based on stacks of protocols, with simple and homogeneous interfaces (in the stack, the protocols communicate events using methods *up()* and *down()*). The applications compose the stacks of protocols as suited and are developed on top of them as a set of blocks. A channel is associated with a protocol stack, which is the concept that allows the association of processes in groups. JGroups provides a set of protocols of different levels, from transport to group management, and even predefined blocks.

3.5 Replication

Understood in a general way, replication encompasses a series of techniques that try to provide greater performance, availability and scalability by maintaining copies of a resource (service or data sets). More particularly, we could distinguish:

Replication itself. A set of nodes, normally constituted in a local network, provides a service. Each node maintains a copy of the service, being able to perform all the functions of the service. Requests to the service are directed to all the nodes or to a subset of them, although *front-end*s can be used as intermediaries. The maintenance of consistency is essential. The main objective is to provide fault tolerance.

Distribution of a service. A set of nodes, normally distributed geographically in different locations, provide a service. Each node maintains a copy of the service, although it may be that not all the functions of the service are replicated in all the nodes. Requests to the service are directed to specific locations. The objectives are to increase availability and scalability, by reducing

[14] http://www.jgroups.org/javagroupsnew/docs/index.html. It was initially called *JavaGroups*.

communication costs[15]. It is used on the Internet (*mirroring*) and in distributed file systems and databases.

Caching. It is the *temporary* storage of subsets of data in the client nodes or in nearby locations. The objective is to increase performance by exploiting the locality. This technique is very common in all types of distributed services. In principle, the management of temporary storage is the responsibility of the client, although the server can collaborate.

	Objectives	Consistency maintenance	Spatial scope	Temporal scope	Management
Replication	Fault tolerance, high availability	Essential	All the system	Permanent	Distributed
Distribution	High availability, low latencies	Secondary	All or part of the system	Permanent	Centralized
Caching	Low latencies	Important	Part of the system	Temporary	In the client

Table 2. Comparison between the different replication techniques.

A comparison of the different replication techniques is shown in Table 2. The same service can use a combination of these techniques. For example, an Internet web server can be supported by a local network of replicated servers, and have also distributed copies in *mirrors* all over the world. In addition, clients accessing the service will temporarily store copies of the most recent pages.

We are going to focus primarily on replication for fault tolerance, which brings together the most general set of concepts and methods. Distribution and caching consider some aspects of replication itself. In particular, caching will be studied within the topic dedicated to distributed file systems.

3.5.1 System model

We will consider a service with a set of clearly specified operations. Clients request operations to the service according to the client-server model. According to the structure of the communication in this model, an operation o_k

[15] In this context, the availability of a service is understood not only as the server that supports it has not failed, but it is able to execute the requests of the clients with bounded latencies. For example, an active but overloaded server may not be available to many clients.

is composed of a pair of invoke-response events (i_k, r_k). Clients are synchronous, that is, given a sequence of operations of the same client, [o_1, o_2, ..., o_k, ...], i_{k+1} only occurs after r_k. We will not make any initial assumption on how the server manages the requests, and in particular whether it is replicated or not, we will see it as a black box.

We will present the model with the help of an example. Consider a system where the possible operations are *Read* and *Write* a value in a variable, with the usual semantic specification: *Write(x, v)* assigns variable x the value v, and *Read(x)* returns the last value assigned to x. Figure 3.3 shows an execution of four operations of two client processes of that system, P_i and P_j. Each process requests a sequence of operations, P_i: [o_a, o_b] and P_j: [o_c, o_d]. For each operation, the value to be written or read is indicated in the chronogram.

If the operations of all the processes are considered globally, a *sequence* is obtained, which we will denote τ, where the particular sequences of each process are interlaced. Since the operations overlap, in the *interlaced sequence* the order of the invocation and response events must be specified. In the example, the resulting interlaced sequence is $\tau = [i_a, r_a, i_c, i_b, r_c, i_d, r_b, r_d]$.

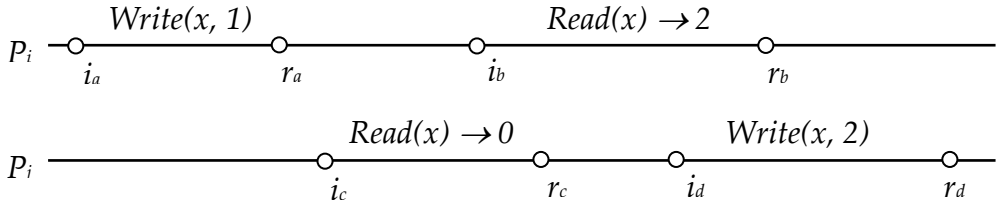


Figure 3.3. Example of execution of a sequence of operations. Initially x=0.

From an interlaced sequence different executions can be obtained. An **execution** is the application of operations in a certain order. A **legal execution**, σ, is one that follows the specification of the operations. In the example, the execution [o_c, o_a, o_d, o_b] is legal, that is[16]:

$$[Read_j(x) \rightarrow 0, Write_i(x, 1), Write_j(x, 2), Read_i(x) \rightarrow 2]$$

and it is not legal

$$[Write_i(x, 1), Read_j(x) \rightarrow 0, ...]$$

[16] Here, the subscripts of the operations indicate the process that performs them.

since the specification of the operations establishes that after $Write_i(x, 1)$, x has value 1, and therefore $Read_j(x) \rightarrow 1$.

We are now interested in defining the consistency criteria that must be set as an objective in a replicated server.

3.5.2 Consistency criteria

The consistency criteria are derived from the restrictions we impose in the order of operations to obtain legal executions from an interlaced sequence. A first consistency criterion of an execution σ with a given interlaced sequence τ is:

Sequential consistency. A legal execution σ is sequentially consistent with τ if, for all processes P_i, the order of P_i's operations in σ matches the order of P_i's events in τ.

In the example of Figure 3.3, the execution

$$[Read_j(x) \rightarrow 0, Write_i(x, 1), Write_j(x, 2), Read_i(x) \rightarrow 2]$$

is sequentially consistent with the defined interlaced sequence, since it is legal and respects the order of the sequences of each process, P_i: [o_a, o_b] and P_j: [o_c, o_d].

As can be seen, a sequentially consistent execution does not have to respect the temporal order in which the operations take place. To make the sequence legal, we had to interpret o_c of P_j as previous to o_a of P_i, although, as shown in the chronogram, the temporal order[17] is the inverse (that is, r_a *precedes* i_c in τ). This interpretation, which may seem unintuitive, can be explained in certain implementations of replicated systems. For example, after P_j writes the value 1 to the variable x, P_i can read x with a value of 0 if it accesses a non-updated replica.

However, in many applications sequential consistency is not enough. For example, in a bank account management system where an operation of depositing an amount into an account and another subsequent charge operation (by a lesser amount) into the same account have been specified, the alteration of the order could lead to a different state if the charge leaves the account in red. The property of transparency in replication requires a stronger form of consistency:

[17] That is, as a global observer would perceive.

Linearizability. A legal execution σ is linearizable if it is sequentially consistent with τ and also, $\forall k, h$, if o_k precedes o_h in σ, then r_h does not precede i_k in τ.

That is, linearizability requires respecting the *global* temporal order in which communication events occur. In the example $Write_i(x, 1)$ must be executed before $Read_j(x)$, since r_a precedes i_c in τ, therefore the execution

$$[Read_j(x) \rightarrow 0, Write_i(x, 1), Write_j(x, 2), Read_i(x) \rightarrow 2]$$

is not linearizable with τ. Instead, the following execution would be linearizable[18]:

$$[Write_i(x, 1), Read_j(x) \rightarrow 1, Write_j(x, 2), Read_i(x) \rightarrow 2]$$

Note that linearizability does not establish anything about the order in which operations that occur concurrently in time must be executed. For example, the following execution would also be linearizable:

$$[Write_i(x, 1), Read_j(x) \rightarrow 1, Read_i(x) \rightarrow 1, Write_j(x, 2)]$$

Linearizability is the consistency criterion that guarantees transparency in replicated systems. The following two conditions are sufficient to ensure linearizability:

- **Order**. Given two requests on an object in a replicated system, all replicas of the system handle these requests in the same order.
- **Atomicity**. Given a request on an object in a replicated system, if a replica of the system handles this request, then all the correct replicas[19] of the system handle this request sooner or later.

3.5.3 Replication techniques

Replication techniques seek a balance between consistency, performance and complexity. Some models (**passive replication** or *primary-backup*) are strongly synchronous, while other models (**active replication**) are based on a stronger group communication support to relax the synchrony and provide a more transparent management of the replicas.

[18] Note that, in order to obtain a linearizable execution, the response values must be modified in the chronogram.

[19] The definition of correct replica is equivalent to the definition of correct process introduced in reliable broadcast.

Linearizability is the general consistency criterion in **fault-tolerant applications**. Sometimes the consistency can be relaxed to minimize the *overhead*, as it is the case of **highly available applications** on the Internet or in distributed file systems, where it is very complicated (or impossible) to provide bounded communication.

3.5.3.1 Passive replication

A node is defined as **primary** and the rest as **secondary**. The client sends the request to the primary, which updates its copy and broadcasts the update to the secondary ones. The secondary send an acknowledgment message to the primary once they have updated their copy. As shown in Figure 3.4, the update of the secondary occurs sequentially with that of the primary.

Since the secondary do not have to deal with concurrent requests, the semantics of the primary-secondary communication to be implemented only needs to be reliable. In effect, the primary node ensures the order condition by sequencing the requests and the atomicity by waiting for the acknowledgment of all the secondary nodes before responding to the client.

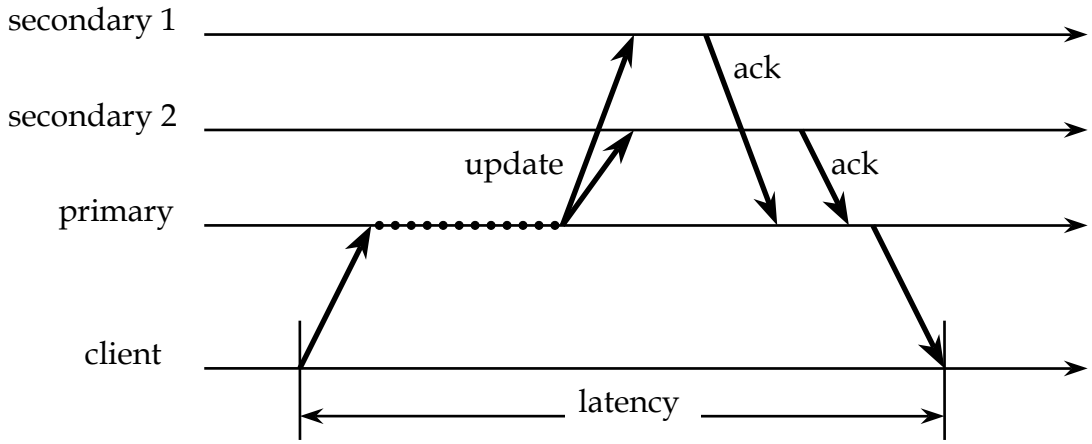


Figure 3.4. Passive replication. Schema of the synchronization between the client and the group of servers.

The failure of a secondary node is detected by the primary when the timeout expires. It is necessary to manage node leaves/joins consistently, and recover the state of a node when it joins the group after recovering from a failure. A failure of the primary implies that one of the secondary nodes is promoted to

primary using an election algorithm as those studied in Section 3.3. In general, a client will re-send the request to the new primary[20] when a timeout expires without receiving a response. The secondary nodes can detect the failure of the primary, for example by means of a heartbeat protocol (periodical message broadcast by the primary indicating that it is working correctly). We must consider different failure situations of the primary node:

(a) The failure occurs outside the interval (invocation, response) of the processing of a request. A secondary node detects the failure and initiates the election protocol.

(b) The failure occurs before broadcasting the update to the secondary nodes. A secondary node detects the failure and initiates the election protocol. The client must re-send the request after a timeout without receiving a response.

(c) The failure occurs after the primary has broadcast the update to the secondary nodes. Reliable broadcast ensures that all or none of the secondary nodes receive the update. In general, it is not possible for the secondary nodes to determine if the client has received the answer or not, so that a further request from the same client could be either a new request or a reinvocation. In this last case, if the secondary nodes did update the state, then the response is simply sent to the client. The problem is solved by labeling in the client the requests with an order number and storing in each node the pairs (invocation, response) of each client.

A variant of passive replication, which relaxes the consistency for unloading the primary, is that secondary nodes receive directly from the clients reading requests, which may not return the latest version. This solution has an interest in applications (for example, content delivery on the Internet) where readings requests are the majority and linearizability is not required. Another alternative of relaxed consistency, aimed at improving latency, is that the primary responds to the client without waiting for the acknowledgment messages of the secondary nodes.

3.5.3.2 Active replication

In active replication a client's request is sent to all replicas, which handle it concurrently. The client is synchronized with the first response obtained and

[20] The identity of the primary node is usually resolved in the name service, which acts as a front-end for the client. The new primary will be registered there after being elected.

discards the responses that it receives later, which will be identical[21]. Figure 3.5 suggests that the concurrency introduced in the accesses to the replicas allows to decrease the latency. However, it is necessary to consider the high cost of supporting the communication semantics (total order) and the fact that the updates are processed in all replicas (dotted lines in Figure 3.5).

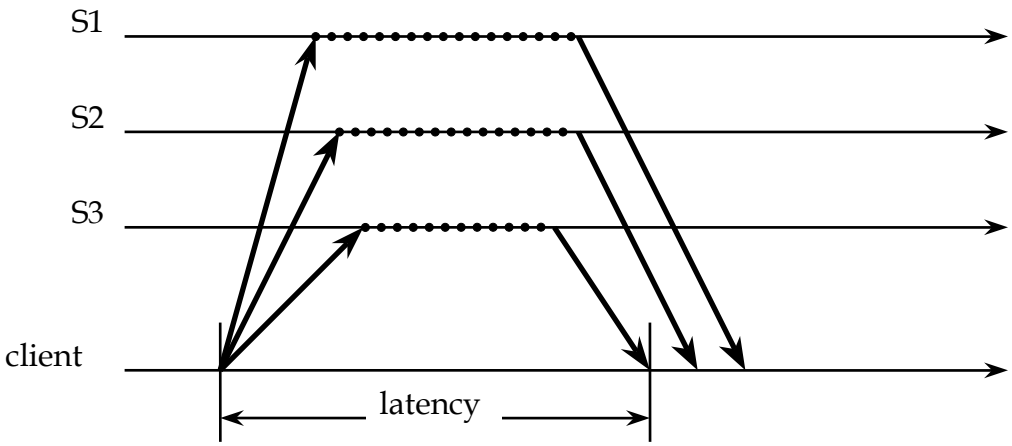


Figure 3.5. Active replication. Schema of the synchronization between the client and the group of servers.

Total order broadcast guarantees the properties of order[22] and atomicity required by linearizability. Unlike in passive replication, where the client must detect and manage the failure of the primary node, in active replication, thanks to the total order broadcast semantics, the failures of the nodes are transparent to the client (provided that all the replicas do not fail).

3.5.3.3 Weak consistency techniques

In environments where communication costs are high, or the number of nodes is very large, the replication models described above do not offer the necessary scalability and latencies become very high. On the other hand, certain applications do not require such strict consistency criteria, as is the case of highly available applications, usually in the scope of the Internet.

An example of these applications is that of a news board geographically distributed and accessible on the Internet. Users can register or unregister, and can read or update news. Users access the distributed system through a *front-*

[21] The service is assumed to be deterministic. If it were not, as for example in a replicated time server, different values would be returned.

[22] We must remember that in our model the clients are synchronous.

end that directs their requests to a local replica. For reading operations it is admissible not to have the latest version, but a causal order should be guaranteed in the updating operations.

The requirement of high availability and bounded latencies[23] in a large-scale environment with high communication costs, such as the Internet, implies that updates are anticipated locally, following simple replication schemes that do not involve the agreement of all the replicas. (**optimistic replication**), which prevents providing strict consistency guarantees. The propagation of updates is done lazily, based on neighborhood criteria. A node performs a synchronization operation with each one of its neighbors following a *peer-to-peer* scheme and executing an algorithm that establishes the order in which updates should be applied, usually based on time vectors (for example, algorithms of anti-entropy [GOL92]).

The criterion for propagating the updates should be such that each update arrives, sooner or later, to the whole system (**eventual consistency**). To provide this property, deterministic algorithms can be used, such as those based on *Distributed Hash Tables* (*DHT*s) [BAL03], or probabilistic algorithms, also called *epidemic* [EUG04].

By its very nature, optimistic replication leads to conflicts (replicas that diverge because the updates are applied in different order.) When a synchronization operation between two replicas detects a conflict, it tries to solve it according to pre-established rules, which may require manual intervention in last instance.

Examples of high availability systems are the *Gossip* architecture (Landin et al, 1992), Bayou (Terry et al, 1995) [COU05 §15] and the Coda file system [SAT90].

3.6 Distributed transactions

Transactions (or atomic transactions) have been used for long in the context of databases to control concurrent access. They are also applicable in distributed file systems and distributed databases (distributed transactions), where each node executes a part of the transaction. It must be noted that in the world of transactions, performance criteria prevail over other considerations, such as transparency.

[23] In this context, high availability and bounded latency mean the same thing: if the system that offers a service is not able to handle requests within a reasonable time, it is considered that it is not available for those clients.

A transaction is a sequence of operations that consistently modifies the state of the system, according to the following properties[24]:

- **Atomicity**. A transaction completes satisfactorily (with agreement) or has no effect on the state of the system.
- **Consistency**. The execution of a transaction preserves the invariants of the system state.[25]
- **Isolation**. The execution of a transaction is independent of the concurrent execution of other transactions.
- **Durability**. The effects of a transaction survive future failures.

The client specifies a transaction through a sequence of operations that includes a set of specific primitives, which in part depend on the type of application to which it is addressed (memory, files, databases). We can consider the following set of abstract primitives:

- *Begin transaction*. Delimits the beginning of a transaction.
- *End transaction*. Delimits the end of a transaction. It originates the attempt of agreement.
- *Abort transaction*[26]. Leave the transaction. It requires that the global state defined before the beginning of the transaction be restored.
- Additional operations for the manipulation of objects, for example: *Reading of an object*, *Writing in an object*.

In what follows, we will focus on the specific scope of distributed transactions.

3.6.1 Execution of distributed transactions

A distributed transaction ensures the consistent update of the global state of the system or the restoration of the previous state if the transaction does not end. A distributed transaction system aims to guarantee the transactional properties (ACID) despite the failure of any of its components. These systems, unlike those

[24] Known by the acronym *ACID*.
[25] The property of consistency is not guaranteed by the transactional system, but by the programmer of the transactions.
[26] The transaction can be aborted at the initiative of the client when this primitive is executed, or at the initiative of a server, as we will see.

based on group communication, always assume a synchronous system model. If a process does not respond within a fixed time bound, it is considered that the process has failed.

We will consider a set of replicated servers that perform the distributed transaction and that we will call **participants**. When a client sends a request to a transactional service, it is received by one of the participants (**initiator**), which communicates the transaction and its start to the rest of participants. One of the participants, the **coordinator**, will initiate a protocol to try to reach an **agreement** on the outcome of the transaction. If there is no agreement, the transaction is **aborted** and a **recovery** mechanism is initiated to restore a stable state. In some systems the initiator is always the same node, in others any node.

Participants exchange messages to perform the transaction and try to agree on the result. We will consider a basic protocol to execute a distributed transaction. Each participant proposes a vote (generically YES or NO) on the result of the execution of the transaction at its node. The coordinator promotes an agreement based on the votes of the participants to ensure that everyone updates or that no one does. The protocol to execute the transaction is as follows:

- The initiator sends to all the participants a start message with the transaction and a period of time to complete it.
- Each participant (including the initiator):
  1. Receives the message to start the transaction.
  2. Executes the operations specified in the transaction.
  3. Depending on the result, establishes its vote, YES or NO, for the update of the global state.
  4. Executes an atomic commitment protocol.

3.6.2 Atomic commitment

The objective of the atomic commitment is to guarantee that the transactional system remains in a consistent global state even in the presence of failures. Basically, an atomic commitment protocol wants each correct process to decide between two alternatives:

- *Commit*. In this case, all the processes participating in the transaction update their state.

- *Abort*. In this case, no process participating in the transaction updates its state.

The decision has the following properties:

1. All participants that decide will do so in the same way.
2. Each participant decides at most once (decisions are irreversible).
3. If a participant decides *Commit*, then all participants have voted YES.
4. If all the participants vote YES and there is no failure, then all the participants decide *Commit*.

A protocol to resolve the atomic commitment requires (at least) two phases: in the first phase, the coordinator receives the votes of the participants. In the second phase, the coordinator communicates the decision to the participants. The following is a generic **two-phase atomic commit protocol** (*2PC*):

Phase 1 (voting)

- The coordinator sends a vote request to all the participants

- Each participant waits for the message from the coordinator. If the message does not arrive in a predefined timeout, the participant assumes that the coordinator has failed and decides *Abort*
- Participants respond by sending their vote[27]

- If all the participants (including the coordinator) send YES, the coordinator decides *Commit*. If not, the coordinator decides *Abort*. If a participant does not respond in a predefined timeout, the coordinator decides *Abort*

Phase 2 (communication of the decision)

- The coordinator sends its decision to all the participants

- Each participant waits for a message with the decision from the coordinator. If the message does not arrive in a predefined timeout, the participant initiates a protocol to try to end the transaction with the help of the rest of participants[28]
- If the message contains the decision of *Commit*, the decision of the participant is *Commit*. If not, the decision of the participant is *Abort*

The 2PC protocol may block in the event of a coordinator failure during the second phase, so more elaborate protocols have been proposed to avoid this problem. [MUL93 §6] presents a more formal description of atomic commitment protocols, providing an analysis of how timeouts are established. The atomic commitment protocols are always synchronous. Atomic commitment is a problem of consensus, and as such its resolution is subject to the restrictions that we will study later.

[27] If a participant has voted NO, he has already decided *Abort*.

[28] Remind that the decision has already been made and cannot be changed. Even the decision could be *Commit*.

3.6.3 Recovery mechanisms

A transactional system must be able to restore the previous state upon a decision of *Abort* (property of atomicity) and to permanently reflect the update in case of *Commit* (property of durability). On the other hand, in databases it is usual to work, for performance reasons, on volatile cache storage. In case of a failure, the updates not backed up in permanent memory would be lost. A general mechanism to avoid this is to keep a history (*logging*) in permanent storage, where, for each transaction, a **list of intentions** that reflects the operations to be performed on the affected data set with its current state is stored.

In distributed transactions it is also necessary to ensure the recovery of a transaction when a node fails during the execution of the atomic commitment protocol. A new intermediate state, *uncertain*, is defined, which is stored in the history when a participant has voted YES and has not received the decision of the coordinator. More information on this subject can be found in [COU05].

3.7 The Consensus problem

Problems such as total order broadcast, group membership management, leader election and atomic commitment have in common that they can be specified as **consensus** problems. In a consensus problem, a decision is made and communicated after collecting the proposals of the participating processes. In abstract terms, the Consensus problem is defined in terms of two primitives:

propose(*v*). The process proposes *v*.

decide(*v*). The process decides *v*.

The Consensus problem is specified by the following properties:

- **Termination**. Every correct process eventually decides some value.
- **Uniform integrity**. Every process decides at most once.
- **Agreement**. No two correct processes decide differently.[29]

[29] The agreement property is *not uniform*. *Uniform agreement* states that decision affects all processes, not just the correct ones: "No two processes, *whether correct or faulty*, decide differently".

- **Uniform validity**. If a process decides v, then v was proposed by some process.[30]

The equivalence between the Consensus problem and the total order broadcast problem can be demonstrated. The other problems cited can also be reduced[31] to the Consensus problem.[32]

According to the impossibility result of Fischer, Lynch and Paterson[33], in asynchronous systems subject to failures, the Consensus problem cannot be solved *deterministically*. This implies the non-resolution in this type of systems of problems reducible to the Consensus problem. The issue is that in an asynchronous system we cannot set time bounds either for the speed of the processes or for the communication delay of a message, so it is not possible to determine if a process that takes time to respond has failed or not.

Synchronous systems do not have this problem, since time bounds can be determined for the transmission delay of a message and the relative speed of the processes (no process can execute more than a finite number S of steps while the slowest process executes 1 step). In fact, in this chapter we have presented synchronous algorithms for leader election, total order broadcast and atomic commitment.

The asynchronous model is the most general. One can pretend to model any system as synchronous by introducing assumptions about the relative speed of processes and communication delays. The problem is where to set these bounds. If they are very conservative, for example by defining long timeouts, applications would suffer in terms of latency upon the failure of a node, which could be unacceptable. If, on the other hand, short timeouts are set, possibly the synchronous system will not be well modeled.

In recent years, alternatives for circumventing the FLP impossibility result have been studied, either through non-deterministic solutions but with a high degree of certainty, or introducing some type of synchrony in the system. For example, within this second approach, Chandra and Toueg addressed the problem by extending the asynchronous system with an **unreliable failure detector**, which monitors the operational status of processes and provides failure information.

---

[30] The validity property captures the relation between the decision value and the proposed values. Changing this property results in other types of Consensus.
[31] The *reduction* of a problem A to another problem B is defined as the resolution of A in terms of B. Two problems are *equivalent* when they can be reduced reciprocally.
[32] Atomic commitment is a stronger problem than Consensus, requiring additional assumptions.
[33] Also known as *FLP impossibility result*.

By using timeouts that increase upon a failure suspicion, the failure detector encapsulates the synchrony, which allows modeling the rest of the system as asynchronous. Each process consults the list of suspected processes provided by its associated failure detector module, and resolves Consensus based on it. As it is unreliable, failure suspicions may be erroneous, but if a model of synchrony with unknown but finite delays is considered, eventually (i.e., in a finite but unknown time) all the timeouts established for the correct processes will be will respected.

[MUL93 §5] provides more details and bibliographic references on the subject.

3.8 Exercises

1 Compare the distributed mutual exclusion algorithms according to the number of messages required to enter and leave the critical section (without and with support for multicast communication). For which number of processes is the algorithm of Ricart and Agrawala profitable compared to the centralized one?

2 Compare the leader election algorithms according to the number of messages exchanged (without and with support for multicast communication).

3 Draw a chronogram that shows an example of non-causal but total order broadcast.

4 Consider a service of distribution lists on the Internet and assume that the broadcast of messages is reliable, but without guarantees on the order of delivery. Discuss how causal broadcast can be provided by a systematic *forwarding* of messages. Compare this technique with the use of time vectors.

5 A family has an online account shared by its members. The son of the family, *S*, operates from Donostia, usually making withdrawals from the account. The parents, *P*, who live in Bilbao, deposit amounts into the account to finance the child's computer engineering studies. The bank offers three types of online operations:

deposit(*cc, q*): $cc = cc + q$

withdraw(*cc, q*): *if* $(cc \geq q)$ $cc = cc - q$; *else ABORT*

balance(*cc*): *returns the value of cc*

Each operation has transactional properties. In particular, once the value of the account (*cc*) has been modified, it remains.

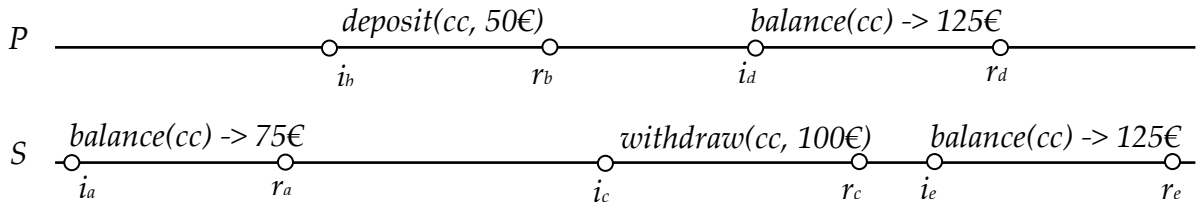
At the beginning of the month, the son needs money to pay his part of rent of the flat, 100€, and in the account there is only 75€, so the son phones to the parents asking to deposit 50€ in the account, what they do immediately, communicating it to the son. The son, once received the notice, transfers the amount of the rent through a withdraw operation. Then, he checks the balance and, seeing that it is 125€, deduces that the withdraw operation has been aborted, so he suspects that the replicated computer system that manages the

online account does not meet the properties that guarantee the desirable consistency semantics (linearizability).

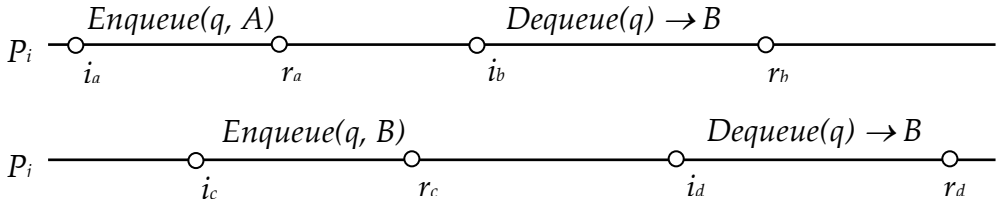
(a) Given the attached chronogram, which shows the sequence of events associated with the operations, which executions are legal?

(b) Regarding the sequence of events, are the executions of the previous section sequentially consistent? Are they linearizable?

(c) We have learned that the online account server has two replicas. Since its behavior is not as expected (the withdraw operation should not have been aborted), it is clear that it is not managed either with an active replication scheme or a passive replication scheme. Consider each scheme as a hypothesis and show that with each scheme the results presented in the chronogram are not possible.



6 Given the following chronogram, in which operations are performed on a FIFO queue implemented on a replicated system with two replicas, is there any possible legal execution for this sequence? If so, is it sequentially consistent? Is it linearizable? Find an interpretation for this result.



7 In the passive replication scheme, analyze the different failure scenarios of the service, indicating in which case(s) the failure transparency, from the point of view of the client, is compromised. How does a service based on active replication behave upon these failure scenarios?

8 Consider the following possible optimizations of the passive replication scheme: (a) the read operations are treated by the secondary nodes (a1) those of the same client by the same secondary, (a2) interchangeably; (b) the primary responds to the client before receiving *ack* from all the secondary nodes. For each possible combination of these optimizations, analyze which consistency criteria are guaranteed and under what conditions.

9 Explain the consequences of implementing passive replication using unreliable broadcast in the communication between the primary and the secondary.

10 In an execution of the timed reliable broadcast algorithm, consider that a process P_1 *R-delivers* three messages at the indicated local time instants. For each message, the process that has broadcast the message and the associated local timestamp is specified. For Δ=10, indicate the local time instants in which P_1 will *TO-deliver* these messages, reasoning why the total order is respected.

1034: m_1 (P_7, $t(m_1)$= 1033)

1034: m_2 (P_5, $t(m_2)$= 1027)

1034: m_3 (P_2, $t(m_3)$= 1033)

11 Show that the following problems are equivalent to the Consensus problem. To do so, express them in terms of the *propose()* and *decide()* primitives.

(a) The delivery with total order of a message broadcasted to a group of processes.

(b) The installation of a new view in a group of processes.

(c) The atomic commitment among the set of processes that execute a distributed transaction.

(d) The election of a process as coordinator.